

Object Oriented Programming (Classes-III)

Abbas Khalid

April 21, 2025

Table of contents

- 1 const Members in Classes
- 2 static Members in Classes
- 3 Friends

const Data Members and Member Functions

- Inside a class, a data member can also be declared as a `const` variable. However, initialization of such class members is done through member initializer list.
- Similarly we can declare `const` member functions inside a class. To declare a member function as `const`, we add a keyword `const` at the end of the function declaration in the class definition i.e.,

```
returnType  functionName(parameter – list) const ;
```

- The keyword `const` must also appear in the function definition.

Example 1

```
class Example{
    int a;
    const int b;
public:
    Example(int i, int j):b(j){a = i;}

    void printA() const
        {cout<<"a = "<< a <<" and b = "<< b <<endl;}

    void printB()
        {cout<<"a = "<< a <<" and b = "<< b <<endl;}
};

int main(){
    Example e(1,2);
    e.printA();
    e.printB();
return 0;
}
```

```
a = 1 and b = 2
a = 1 and b = 2
```

Example 2

```
class Example{
    int a;
    const int b;
public:
    Example(int i, int j):b(j){a = i;}

    void printA() const;

    void printB()
    {cout<<"a = "<< a <<" and b = "<< b <<endl;}
};

int main(){
    Example e(1,2);
    e.printA();    e.printB();
return 0;
}
```

a = 1 and b = 2
a = 1 and b = 2

```
void Example::printA() const
{cout<<"a = "<<a<<" and b = "<<b<<endl;}
```

const Objects

- We can also declare `const` objects of a class.
- Passing parameters by references or pointers avoids unnecessary copying of objects, enhancing performance but risking unintended modifications to the original object beyond the function's scope. Using `const` in parameter declarations prevent unintended modifications to the received object.
- For any object that we declare to be `const`, we can only call member functions that are also declared to be `const`. A `const` member function does not modify the object for which it is called. Actually, if we specify an object as `const`, we are telling the compiler that we will not modify it.
- When we call a function that is not `const` using a `const` object, the compiler will complain, because it has no guarantee that the function will not modify the object. However, the function declared as `const` also work for non-`const` objects.
- `const` cannot be used for constructors and destructors. The job of a constructor is to initialize field values, so it must change the object. Similarly for destructors.
- `const` can be overloaded with non-`const` functions. The compiler will choose the appropriate one to call depending on the object they are being applied to.

Example 1

```
class Example{
    int a;
    const int b;
public:
    Example(int i, int j):b(j){a = i;}

    void printA() const
        {cout<<"a = "<< a <<" and b = "<< b <<endl;}

    void printB()
        {cout<<"a = "<< a <<" and b = "<< b <<endl;}
};

int main(){
    const Example e(1,2);
    e.printA();    // OK
    e.printB();    //Error!const object & non-const function
return 0;
}
```

Example 2

```
class Printer{
public:
    void print() const
        {cout<<"A const member function"<<endl;}
    Printer(){} // Explicitly Required?
    Printer(const Printer& P)
        {cout<<"Copy Constructor"<<endl;}
};

void printObject(const Printer& obj) {
    obj.print(); // Call to const member function
}

int main() {

    const Printer obj; // Const object
    printObject(obj); // Passing const object to function

    return 0;
}
```

A const member function

Example 3

```
class Overload{
public:
    void print()const{cout<<"Const Function"<<endl;}

    void print(){cout<<"Non-const Function"<<endl;}
};

int main(){

    const Overload obj1; // Const object
    Overload obj2;       // Non-const object

    obj1.print(); //Calls the const print
    obj2.print(); //Calls the non-const print

    return 0;
}
```

Const Function
Non-const Function

Exercise 1

```
class Demo{
    int i;
public:
    Demo(int a):i(a){}
    int getI()const{return i;}
};

int main(){
    const Demo obj1(10);
    Demo obj2 = obj1;
    cout<<obj2.getI()<<endl;

    return 0;
}
```

```
class Demo{
    int i;
public:
    Demo(int a):i(a){}
    int getI()const{return i;}
};

int main(){
    Demo obj1(10);
    const Demo obj2 = obj1;
    cout<<obj2.getI()<<endl;

    return 0;
}
```

10

Exercise 2

```
class Demo{
    int i;
public:
    Demo(int a):i(a){}
    int getI()const{return i;}
    Demo(const Demo& obj)
    {cout<<"Copying.."<<endl;}

};

int main(){
    const Demo obj1(10);
    Demo obj2 = obj1;
    cout<<obj2.getI()<<endl;

return 0;
}
```

```
class Demo{
    int i;
public:
    Demo(int a):i(a){}
    int getI()const{return i;}
    Demo(const Demo& obj)
    {cout<<"Copying.."<<endl;}

};

int main(){
    Demo obj1(10);
    const Demo obj2 = obj1;
    cout<<obj2.getI()<<endl;

return 0;
}
```

Copying..
0

Exercise 3

```
class Demo{
    int i;
public:
    Demo(int a):i(a){}
    int getI()const{return i;}
    Demo(const Demo& obj)const
    {cout<<"Copying.."<<endl;}
};

int main(){
    const Demo obj1(10);
    Demo obj2 = obj1;
    cout<<obj2.getI()<<endl;

    return 0;
}
```

```
class Demo{
    int i;
public:
    Demo(int a)const {i = a;}
    int getI()const{return i;}
    Demo(const Demo& obj)
    {cout<<"Copying.."<<endl;}
};

int main(){
    Demo obj1(10);
    const Demo obj2 = obj1;
    cout<<obj2.getI()<<endl;

    return 0;
}
```

Mutable Data Members of a Class

- By default, `const` member functions in C++ are not allowed to modify any member variables of the class. However, there are situations where we might need to modify certain member variables. In such cases, we can declare those member variables as mutable.
- We use the `mutable` keyword to specify that a particular member variable can be modified even within `const` member functions of the class. This allows to modify the state of the object.
- The `mutable` keyword applies only to member variables and cannot be used with non-member (or local) variables. It controls their mutability within `const` member functions.
- The `mutable` is often used for caching or memoization purposes. For example, if a member function calculates a value and stores it in a member variable for future use, that member variable can be declared as `mutable` so that it can be modified even in `const` member functions, allowing the caching mechanism to work properly.

Example 1

```
class Demo{
    int i;
public:
    Demo(int a):i(a){}
    int get()const{return i;}
    void set(int x)const{i = x;} //Modify using const func
};

int main(){

    const Demo obj(1);
    obj.set(1974); //Error!
    cout<<obj.get();

    return 0;
}
```

Example 2

```
class Demo{
    mutable int i;
public:
    Demo(int a):i(a){}
    int get()const{return i;}
    void set(int x)const{i = x;} //Modify using const func
};

int main(){

    const Demo obj(1);
    obj.set(1974); //OK
    cout<<obj.get();

    return 0;
}
```

1974

Example 3

```
class Demo{
    int i;
    mutable int j;
public:
    Demo(int a, int b):i(a),j(b){}
    int getI()const{return i;}
    int getJ()const{return j;}
    void set(int x, int y)const{i = x;j = y;}
};

int main(){

    const Demo obj(1,2);
    obj.set(5, 10); //Error!
    cout<<obj.getI()+ obj.getJ();

    return 0;
}
```


Example 4

```
class Demo{
    int i;
    mutable int j;
public:
    Demo(int a, int b):i(a),j(b){}
    int getI()const{return i;}
    int getJ()const{return j;}
    void set(int x, int y)const{j = y;}
};

int main(){

    const Demo obj(1,2);
    obj.set(5, 10); //OK
    cout<<obj.getI()+ obj.getJ();

    return 0;
}
```

11

static Members in Classes

- Each object of a class has its own copy of all the data members of the class. However, if we require only one copy of a variable shared by all objects of a class, a `static` class variable is used.
- A `static` class variable represents class-wide information and it can be accessed by any object of the class.
- Both data members and functions of a class can be declared as `static`.
- Constant member functions promise not to modify the state of the object on which they are called, but they don't prevent modifications to `static` data members or other global state outside of the object.

static Data Members

- When we precede a member variable's declaration with `static`, we are telling the compiler that only one copy of the variable will exist and that all objects of class will share that variable. The `static` data member is defined only once and will exist even if no objects of the class have been created.
- When we declare a `static` data member within a class, we cannot define it (i.e., no storage can be allocated here). Defining a `static` data member needs 2-steps:
 - 1 declare the member in class declaration using following statement:
`static dataType varName;`
 - 2 define the data member globally outside the class as follows:
`dataType className::varName;`
- In the second step, we can also initialize it with a certain value.
`dataType className::varName = value;`
- By default, all `static` variables are initialized to zero.

static Data Members

- The binary scope resolution operator (::) is used to identify the class to which the `static` variable belongs.
- The `static` data members can be `public`, `private` or `protected`.
- A class's `public static` member can be accessed through any object of that class.
- To access a `public static` member when no object of the class exist, we prefix the class name and the scope resolution operator to the name of the data member.
- A class's `private` and `protected static` members must be accessed through `public` member function of the class or through `friend` of the class.
- To access a `private` or `protected static` class member when no object of the class exist, a `public static` member function must be provided and the function must be called by prefixing its name with the class name and scope resolution operator.

Example 1

```
class Shared{  
    public:
```

```
        static int a; //Declaration
```

```
};
```

```
    int Shared::a; //Definition
```

```
int main(){  
    //Initialize before creating any object
```

```
    Shared::a = 99;
```

```
    cout<<"Using class name ="<<Shared::a<<endl;
```

```
    Shared x; //Creating object
```

```
    cout<<"Using class object ="<<x.a<<endl;
```

```
    return 0;  
}
```

```
Using class name = 99  
Using class object = 99
```

Example 2

```
1 class Shared{
2     static int a;
3     int b;
4 public:
5     void set (int i, int j){a = i; b = j;}
6     void show();
7 };
8
9 int Shared :: a; // by default zero
10 void Shared :: show(){
11     cout<<"Static a ="<<a<<endl;
12     cout<<"Non-static b ="<<b<<endl;
13 }
14
15 int main(){
16     Shared x, y;
17     x.set(1,1);  x.show();
18     y.set(2,2);  y.show();
19     x.show();    // "a" is shared by both objects
20 return 0;
21 }
```

Static a = 1
Non-static b = 1
Static a = 2
Non-static b = 2
Static a = 2
Non-static b = 1

Example 3

```
1 class Resource{
2     static int resource;
3 public:
4     int get();
5     void free(){resource = 0;}
6 };
7
8 int Resource::resource;
9 int Resource::get(){
10     if(resource) return 0;    //Resource is in use
11     else{resource = 1; return 1;} //Resource allocated now
12 }
13
14 int main(){
15     Resource ob1, ob2;
16     if(ob1.get())cout<<"Resource assigned to Ob1."<<endl;
17     if(!ob2.get())cout<<"Resource denied to Ob2."<<endl;
18     ob1.free();    //Let someone else use it
19     if(ob2.get())cout<<"Resource assigned to Ob2."<<endl;
20 return 0;
21 }
```

Resource assigned to Ob1.
Resource denied to Ob2.
Resource assigned to Ob2.

const static & static const Data Members

```
1  //Special Case: A data member that is both static and const, i.e.
2  //it's shared among all instances of class and cannot be modified after initialized.
3  class Test {
4      public:
5          // Declaring and initializing a static data member as const
6          static const int staticConst = 20;
7
8          // Declaring and initializing a const data member as static
9          const static int constStatic = 10;
10 };
11
12 int main() {
13     Test t;
14     // Accessing the const static member
15     cout<<"Static Const value: "<<Test::staticConst<<endl;
16     cout<<"Static Const value: "<<t.staticConst<<endl;
17     // Accessing the const static member
18     cout<<"Const Static value: "<<Test::constStatic<<endl;
19     cout<<"Const Static value: "<<t.constStatic<<endl;
20     return 0;
21 }
```

Static Const value: 20
Static Const value: 20
Const Static value: 10
Const Static value: 10

const static & static const Data Members (2)

```
1 class Test {
2     public:
3         // Declaring a static data member as const
4         static const int staticConst;
5
6         // Declaring a const data member as static
7         const static int constStatic;
8 };
9 // Initialization
10 const int Test::staticConst = 20;
11 const int Test::constStatic = 10;
12 int main() {
13     Test t;
14     // Accessing the const static member
15     cout<<"Static Const value: "<<Test::staticConst<<endl;
16     cout<<"Static Const value: "<<t.staticConst<<endl;
17     // Accessing the const static member
18     cout<<"Const Static value: "<<Test::constStatic<<endl;
19     cout<<"Const Static value: "<<t.constStatic<<endl;
20     return 0;
21 }
```

Static Const value: 20
Static Const value: 20
Const Static value: 10
Const Static value: 10

static Member Functions

- Member functions may also be declared as **static**. A **static** member function exists even if no class objects have been created. There are several restrictions placed on the **static** member functions:
 - They can refer to other **static** members of the class only.
 - A **static** member function does not have a **this** pointer.
 - There cannot be a **static** and a non-**static** version of the same function.
 - A **static** member function may not be **virtual**.
 - They cannot be declared as **const** or **volatile**.
- To access a **static** member function when we have no object defined, we use:

```
className :: funName();
```

- To access a **static** member function with an object, we use:

```
objName.funName();
```

Example 1

```
1 class ST{
2     static int i;
3 public:
4     static void init(int x){i = x;}
5     void show(){cout<<i<<endl;}
6 };
7
8 int ST::i;
9
10 int main(){
11
12     ST::init(100);
13     ST x;
14     x.show();
15
16 return 0;
17 }
```

100

Example 2

```
1 class MyClass {
2     static int totalObjects; // Total objects
3     int id; // To assign a unique ID to each instance
4 public:
5     MyClass() {
6         totalObjects++; // Records total objects created
7         id = totalObjects; // Assign an ID to each instance
8     }
9     void display() const { cout << "Instance ID:" << id << endl; }
10    static int getTotal() { return totalObjects; }
11 };
12 int MyClass::totalObjects = 0;
13 int main() {
14     MyClass obj1, obj2, obj3;
15     obj1.display();
16     obj2.display();
17     obj3.display();
18     cout << "Total objects:" << MyClass::getTotal() << endl;
19     return 0;
20 }
```

Instance ID:1
Instance ID:2
Instance ID:3
Total objects:3

Example 3

```
1 // SharedState class to encapsulate the shared state
2 class SharedState {
3     static int sharedValue; //Member to hold shared state
4     public:
5         static void setValue(int value){sharedValue = value;}
6         static int getValue(){return sharedValue;}
7 };
8
9 int SharedState::sharedValue = 0; //Initialize static member
10
11 class A {
12     public:
13         A(int val) {SharedState::setValue(val);}
14
15         void print() const
16             {cout<<"Value in A:"<<SharedState::getValue()<<endl;}
17
18         // Update the shared state
19         void modify(int newVal){SharedState::setValue(newVal);}
20 };
```

Example 3 (Cont.)

```
1  class B {  
2      public:  
3          B(int val) {SharedState::setValue(val);}  
4  
5          void print() const  
6              {cout<<"Value in B:"<<SharedState::getValue()<<endl;}  
7  
8          void modify(int newVal){SharedState::setValue(newVal);}  
9  };  
10  
11  int main() {  
12  
13      A objA(5); B objB(10);  
14      objA.print(); objB.print();  
15  
16      objA.modify(20);  
17      objA.print(); objB.print();  
18  
19      return 0;  
20  }
```

Value in A: 10
Value in B: 10
Value in A: 20
Value in B: 20

static Objects

- When an object is declared as **static** within a function or at global scope, it has static storage duration. This means that the object persists for the entire duration of the program. It is initialized only once, before the program starts executing, and remains alive until the program terminates.
- Unlike local variables, which are created and destroyed each time their enclosing block is entered and exited, a **static** object persists throughout the program's execution. It is initialized before any function in the program is called and destroyed after the program terminates.
- If the **static** object is declared within a function, it is accessible only within that function. If it's declared at the global scope, it is accessible throughout the entire program.
- Static objects are allocated memory in the static storage area, separate from the stack and heap. This memory allocation occurs at compile time.
- Static objects can be useful for scenarios where you need a single instance of a class to be shared across multiple parts of your program.

Example 1

```
1 class Test {
2     public:
3         Test() {cout <<"Constructor executes\n"; }
4         ~Test() {cout <<"Destructor executes\n"; }
5 };
6
7 int main() {
8
9     cout <<"main() starts\n";
10    {
11        Test t1;    // Ordinary Object (Local)
12    }
13    cout <<"main() terminates\n";
14
15    return 0;
16 }
```

main() starts
Constructor executes
Destructor executes
main() terminates

Example 2

```
1 class Test {
2     public:
3         Test() {cout <<"Constructor executes\n"; }
4         ~Test() {cout <<"Destructor executes\n"; }
5 };
6
7 int main() {
8
9     cout <<"main() starts\n";
10    {
11        static Test t1;    // Static Object (Local)
12    }
13    cout <<"main() terminates\n";
14
15    return 0;
16 }
```

main() starts
Constructor executes
main() terminates
Destructor executes

Example 3

```
1 class Test {
2     public:
3         Test() {cout <<"Constructor executes\n"; }
4         ~Test() {cout <<"Destructor executes\n"; }
5 };
6
7 Test t1;    // Ordinary Object (Global)
8
9 int main() {
10
11     cout <<"main() starts\n";
12     cout <<"main() terminates\n";
13
14     return 0;
15 }
```

Constructor executes
main() starts
main() terminates
Destructor executes

Example 4

```
1 class Test {  
2     public:  
3         Test() {cout <<"Constructor executes\n"; }  
4         ~Test() {cout <<"Destructor executes\n"; }  
5 };  
6  
7 static Test t1;    // Static Object (Global)  
8  
9 int main() {  
10  
11     cout <<"main() starts\n";  
12     cout <<"main() terminates\n";  
13  
14     return 0;  
15 }
```

Constructor executes
main() starts
main() terminates
Destructor executes

Example 5

```
1 class Singleton {
2     Singleton() {} // Private constructor
3     static Singleton instance; // Static instance
4 public:
5     static Singleton& getInstance() {return instance;}
6     void someMethod(){cout<<"Singleton method"<<endl;}
7 };
8
9 Singleton Singleton::instance;
10
11 void functionUsingSingleton() {
12     Singleton& singletonInstance = Singleton::getInstance();
13     singletonInstance.someMethod();
14 }
15
16 int main() {
17     functionUsingSingleton();
18     functionUsingSingleton();
19     return 0;
20 }
```

Singleton method
Singleton method

Friends

- Usually we hide the data members of our class by declaring them as `private` so that any function that is not a member of this class cannot access these data members.
- However, there are some honorary functions that are not members of a class but still they can access non-`public` members of the class. Such functions are called `friends` of the class.
- A `friend` can access any of the members of a class object, regardless of their access specification.
- An individual function can be specified as a `friend` of a class as well as a whole class can be specified as a `friend` of another class.
- One of the distinguishing features of C++ is `encapsulation` i.e. bundling of data and functions operating on that data together so that no outside function or class can access the data. But by allowing the `friend` functions or class to access the `private` members of another class, we actually compromise the `encapsulation` feature.

Friend Functions

- A function that is not member of a class but can access all its members is called a *friend* function.
- To make a function *friend* of a class, we must declare it within the class definition using the keyword *friend*. The general form is:

```
1 class className{  
2 :  
3 friend returnType functionName(parameter-list);  
4 };
```

- Note that a function cannot be *friend* of a class of which it is a member and the access specifiers do not apply to the friends of a class.
- Usually object of the class (that declares a function as its *friend*) is passed through the *friend* function as a parameter.

Friend Functions

- The *friend* function can be defined anywhere in the code file and we need not use the keyword *friend* or the scope resolution operator.
- A *friend* function can be declared in the *private* or *public* section of the class. It can be called like a normal function without using the object.
- A *friend* function is not in the scope of the class, of which it is a friend.
- A *friend* function is not invoked using the class object as it is not in the scope of the class.
- A *friend* function cannot access the private and protected data members of the class directly. It needs to make use of a class object and then access the members using the dot operator.
- A *friend* function can be a global function or a member of another class.

Example 1

```
class F{  
    int a,b;  
public:  
    F(int i, int j){a = i; b = j;}  
  
    friend int A(F x){return x.a + x.b;} //Not a member  
};  
  
int main(){  
  
    F k(3,4);  
    cout<<A(k);  
  
    return 0;  
}
```

7

Example 2

```
class F{  
    int a,b;  
public:  
    F(int i, int j){a = i; b = j;}
```

```
    friend int A(F x); // friend function
```

```
};
```

```
    int A(F x){return x.a + x.b;} //Not a member
```

```
int main(){  
  
    F k(3,4);  
    cout<<A(k);  
  
    return 0;  
}
```

7

Friend Classes

- The general form to declare a *friend* class is:

```
1 class A{  
2 :  
3 friend class B;  
4 };  
5 class B{  
6 :  
7 };
```

- Here class *B* is a *friend* of class *A*. So class *B* can access the *private* and *protected* members of class *A*.
- This does not mean that class *A* can access *private* and *protected* members of the class *B* i.e. the friendship is not mutual unless we make it so.
- Similarly, the friendship of the class is not inherited. This means that as class *B* is a friend of class *A*, it will not be a *friend* of the subclasses of class *A*.

Example 1

```
class Two{
    int a,b;
public:
    Two(int I, int j){ a = I; b = j;}

    friend class Min; //Declaring a friend class
};

class Min{
public:
    int min(Two x){return x.a < x.b ? x.a : x.b;}
};

int main(){
    Two obj(10, 20);
    Min m;
    cout<<m.min(obj);

    return 0;
}
```

10

Example 2

```
class Two; // Forward Declaration is required
class Min{
public:
    // Definition not allowed here.
    // int min(Two x) {return x.a < x.b ? x.a : x.b;}
    int min(Two x); // Prototype Allowed
};
class Two{
    int a,b;
public:
    Two(int I, int j){ a = I; b = j;}
    friend class Min; // Declaring a friend class
};
int main(){
    Two obj(10, 20);    Min m;    cout<<m.min(obj);
return 0;
}
int Min::min(Two x){return x.a < x.b ? x.a : x.b;}
```

10

Example 3

```
class B;    // Forward Declaration
class A{
    friend class B;    int a;
public:
    A(int x):a(x){}
    // void display(B ob){cout<<ob.b;}--Incorrect
    // B is not defined yet, so ob.b does not exist.
    void display(B ob); // Only declaration
};

class B{
    friend class A;    int b;
public:
    B(int y){b = y;}
    void display(A ob){cout<<"Class B : "<<ob.a<<endl; }
};

int main() {
    A a(5); B b(10); a.display(b); b.display(a);
return 0;
}

//Definition goes here. Class B has been defined now.
void A::display(B ob){cout<<"Class A : "<<ob.b<<endl;}
```

Class A:10
Class B:5

Example 4

```
class ClassB; // Forward declaration
class ClassA {
    int numA;
    friend int add(ClassA, ClassB);
public:
    ClassA() : numA(12) {}
};
class ClassB {
    int numB;
    friend int add(ClassA, ClassB);
public:
    ClassB() : numB(1) {}
};
int add(ClassA objectA, ClassB objectB)
{return (objectA.numA + objectB.numB);}
int main() {
    ClassA objectA; ClassB objectB;
    cout << " Sum: " << add(objectA, objectB);
return 0;
}
```

Sum: 13

Exercise: What will be the Output?

```
class A{
    int a;
public:
    A(int a1 = 1){a = a1;}
    void display(){cout<<" Class A.."<<endl;}
    friend class B;
};
class B{
    int b;
public:
    B(int b1 = 1){b = b1;}
    int add(A ob){return (ob.a + b);}
    void display(){cout<<"Class B.. " <<endl;}
    int get(){return b;}
};
int main() {
    A a(10); B b(20);
    a.display(); b.display();
    cout<<"b.add(a) ="<<b.add(a)<<endl;
return 0;
}
```

